

GuanDan V2.0 项目功能说明

面向线下攒蛋赛事的报名导入、自动对阵、录分统计与大屏展示系统

项目项	说明
项目目录	E:\Pycharm-pro\my_project\GuanDan\GuanDan_V2.0
文档类型	项目功能说明 / 交接说明
生成日期	2026-05-12
技术栈	前端 Vue 3 + Vite; 后端 Flask; 数据层 MySQL; 缓存/异步写回 Redis
阅读提示 本文根据当前仓库中的前后端代码、接口规范和算法说明整理，重点说明系统已经承载的业务功能、主要模块边界和运行依赖，便于项目展示、交接和后续迭代。	

1. 项目定位

GuanDan V2.0 是一个面向线下攒蛋比赛组织的赛事管理与展示系统。项目覆盖从报名 Excel 导入、队伍与等级初始化、三轮对阵生成、比赛轮次控制、成绩录入、排行榜统计，到前端大屏轮询展示的完整业务闭环。

系统当前采用“后台管理 + 前端大屏展示”的形态：后端保留 Flask 模板页面用于导入、生成对阵、设置轮次和录分；前端 Vue 应用通过聚合快照接口获取展示数据，用于现场屏幕展示当前桌位、队伍分数、办公室排名、队伍排名和倒计时。

1.1 核心价值

- 减少人工排表成本：系统根据队伍、办公室和等级信息自动生成多轮对阵。
- 支撑现场实时展示：前端按固定间隔拉取快照数据，保持大屏内容刷新。
- 统一成绩口径：录分后自动计算大小分，并按队伍和办公室维度统计排名。
- 兼容比赛现场变更：支持设置当前轮次、启动/停止计时、修改指定轮次和桌号的比分。
- 为扩展预留基础：后端已拆出部分 services，并引入 Redis 缓存和写回队列以降低查询与写入压力。

2. 总体架构

项目分为前端展示端和后端服务端。前端目录为 GuanDanFront-V2.0，主要负责赛事大屏、座位图、动态分组结果展示和数据轮询；后端目录为 GuanDan_backend-V2.0，负责业务数据导入、对阵生成、录分、统计查询、缓存刷新和模板管理页面。

层级	主要目录 / 文件	职责
前端展示层	GuanDanFront-V2.0/src	Vue 3 单页应用，展示比赛状态、桌位对阵、排行榜、二维码、座位图和动态分组结果。
接口访问层	src/api.js, src/composables/useGuandan Data.js	读取环境变量中的后端地址，调用 /dashboard_snapshot 聚合接口并规范化数组/对象数据。
后端路由层	run.py, api/frontend_api.py	提供模板管理页面、JSON 展示接口和聚合快照接口。
业务服务层	services/*.py	拆分导入、对阵、录分、统计、缓存、日志和异步写回等业务能力。
数据层	MySQL + Redis	MySQL 保存报名、对阵和成绩；Redis 用于对阵缓存、仪表盘快照辅助和可选成绩写回队列。

3. 业务流程

步骤	环节	系统行为
1	报名导入	管理员上传 .xlsx 报名表，系统解析办公室、队长、四支子队、等级、成员和替补信息，并初始化三轮成绩。
2	对阵生成	系统读取队伍、办公室和等级信息，生成三轮对阵并写入 fight_info。
3	轮次控制	管理员设置当前轮次，启动或暂停 60 分钟倒计时。
4	现场展示	前端大屏轮询 /dashboard_snapshot，显示当前轮次、倒计时、桌位对阵、当前分数和排行榜。
5	成绩录入	录分人员按桌号进入录分页面，输入双方最终等级，系统计算大小分并写入成绩表。
6	统计更新	后端标记快照过期，后台线程刷新聚合数据，前端下一轮轮询后显示最新排名。
7	赛后修正	后台可按轮次和桌号重新进入录分页面修改比分，保留操作日志用于追溯。

4. 功能模块说明

4.1 报名 Excel 导入与初始化

后台首页支持上传 .xlsx 报名表。导入服务从第三行开始读取数据，按字段写入 team_info、team_level、mini_team_info 和 office_info 等表。每个办公室最多包含四支子队，系统会为每支有效子队初始化三轮 big_score 和 small_score。

- 校验文件后缀必须为 .xlsx，并使用 secure_filename 处理上传文件名。
- 队员字段会做格式归一化，便于后续对阵和前端展示。
- 导入成功后会清理对阵缓存、重置当前轮次并标记大屏快照过期。

4.2 对阵生成

对阵生成是系统核心功能。当前后台生成对阵时调用 generate_round_pairs(..., rounds=3)，按三轮比赛生成配对结果，并通过 replace_fight_info 覆盖写入 fight_info。算法目标是保证每轮每队只出现一次、同办公室不互打、尽量避免重复对阵，并尽量覆盖不同等级组合。

代码中还保留了 generate_round_pairs_large_scale，用于更大规模场景。该版本采用稀疏候选、贪心匹配、局部修复和分级放宽策略，理论复杂度接近 $O(\text{rounds} * N * \text{candidate_k})$ ，适合在队伍数量增加时作为后续升级方向。

4.3 轮次、计时与播放控制

后端维护当前轮次 TURN，合法值为 1、2、3 或 null。管理员可通过后台设置当前轮次，启动或停止倒计时。倒计时总长为 60 分钟，前端显示 mm:ss 格式；未启动时显示未开始状态。前端还包含五分钟提醒音频逻辑，用于现场时间提示。

4.4 成绩录入、改分与日志

录分流程以当前轮次和桌号为入口。系统读取该桌双方队伍和成员信息，录入双方最终等级后，通过 build_score_update 计算小分差和大分归属。若双方最终等级相同，需要额外指定最后一局赢家，作为大分判定依据。

- 等级输入范围为 2 到 32。
- 胜方大分为 2，负方大分为 0；小分为双方等级差值。
- 支持按指定轮次和桌号修改比分。
- 录分成功后保存 score log，并标记仪表盘快照过期。
- 可通过 SCORE_WRITEBACK_ENABLED 启用 Redis 写回队列，批量压缩后落库。

4.5 统计排名与大屏展示

前端展示端主要服务于比赛现场大屏。应用启动后每隔固定时间调用 /dashboard_snapshot，一次获取当前轮次、倒计时、桌位对阵、当前小分、办公室排名和队伍排名。排行榜同时展示当前轮得分和累计得分。

展示组件	说明
首页 HomeScreen	当前轮次未开始时展示活动首页。
计时 TimerDisplay	展示倒计时，并支持点击触发音频测试。
桌位 GameTables	以桌卡形式展示桌号、两队队名、成员和当前小分。
排名 Rankings	展示办公室维度和队伍维度的当前轮排名、累计排名。
二维码 QRCode	展示扫码计分入口二维码。
座位图 SeatingArrangement	全屏展示座位安排图片。
动态分组	支持上传 Excel 或使用默认 Excel 进行动态分组预览、统计与导出。

4.6 缓存与聚合快照

后端为大屏查询设计了聚合快照机制。后台线程定时刷新对阵、比分、队伍排名和办公室排名，/dashboard_snapshot 直接返回快照副本，减少前端多接口轮询和数据库重复查询。数据变化时通过 mark_snapshot_stale 将快照标记为过期，下次请求或后台刷新会更新数据。

- 对阵信息读取路径为本地进程缓存 -> Redis -> MySQL。
- 前端主流程优先使用 /dashboard_snapshot，旧的 /matchesinfo、/scoresinfo、/sumteaminfo 等接口保留兼容。
- 队伍排名在计时开始后按 5 秒一段做切片轮播，每段最多 6 条。

5. 主要接口

接口	方法	用途
/	GET/POST	后台首页；展示报名数据或上传 Excel 导入。
/clear_tables	POST	清空业务表、清理缓存并重置当前轮次。
/set_turn	POST	设置当前轮次。
/start_timer	POST	启动当前轮次倒计时。
/stop_timer	POST	停止倒计时。
/generate_matches	GET/POST	展示或生成三轮对阵。
/select_table	GET/POST	按当前轮次选择桌号并进入录分。
/modify_select_table	GET/POST	按轮次和桌号进入改分。

接口	方法	用途
/input_scores/	GET/POST	展示录分页并提交比分。
/dashboard_snapshot	GET	前端大屏聚合快照接口。
JSON 接口	返回内容	
/TURNsinfo	当前轮次 TURN。	
/timesinfo	倒计时文本。	
/matchesinfo	当前轮次对阵信息。	
/scoresinfo	当前轮次小分信息。	
/sumteaminfo	队伍当前轮和累计排名。	
/officescore	办公室当前轮和累计排名。	
/dashboard_snapshot	聚合返回轮次、倒计时、对阵、比分、队伍排名、办公室排名和快照更新时间。	

6. 数据表与数据含义

数据表	主要用途
team_info	保存报名原始信息，包括办公室、队长、子队和成员。
team_level	保存每支子队的等级标签，用于对阵生成。
mini_team_info	保存每支子队在每一轮的大分、小分，是统计排名的核心表。
office_info	保存办公室维度的轮次初始化数据，办公室排名也可由 mini_team_info 聚合得到。
fight_info	保存每轮桌号与双方队伍、成员的对阵关系。

7. 运行与部署依赖

后端依赖 Flask、flask-cors、mysql-connector-python、openpyxl、redis、pandas 等 Python 包；前端依赖 Vue 3、Vite 和 xlsx。数据库默认连接到 MySQL 的 guan_egg_game，Redis 默认连接本机 6379，可通过环境变量覆盖。

配置项	默认值	说明
APP_HOST	0.0.0.0	后端监听地址。
APP_PORT	5000	后端端口。
DB_HOST / DB_PORT	127.0.0.1 / 3306	MySQL 地址与端口。
DB_NAME	guan_egg_gam	业务数据库名。

配置项	默认值	说明
	e	
REDIS_HOST / REDIS_PORT	127.0.0.1 / 6379	Redis 地址与端口。
SCORE_WRITEBACK_ENABLED	1	是否启用 Redis 成绩写回队列。
VITE_BASE_API / VITE_BASE_PORT	由 .env 提供	前端请求后端的基础地址和端口。

8. 当前限制与后续建议

当前系统已经具备完整比赛闭环，但仍处于迭代型工程状态。若后续用于长期运维或多人协作，建议优先处理接口规范、状态持久化、输入校验和编码一致性。

- 将模板页面操作逐步补齐为 /api/v1/* JSON 接口，前端无需处理 redirect 与 flash。
- 将当前轮次、计时器状态等进程内变量迁移到 Redis 或数据库，避免多进程部署状态不一致。
- 梳理并统一源码文件编码为 UTF-8，避免中文注释和界面文字在不同环境下显示异常。
- 为导入、对阵生成、录分计算和统计排名增加固定测试数据与自动化测试。
- 将同办公室不互打、重复对阵、等级均衡等规则参数化，便于不同赛事灵活调整。
- 完善后台权限控制，避免导入、清空、改分等高风险操作默认开放。